

```

import { q, one, j } from '../db.js';
import { getCapability, resolveDisposition } from './policy.js';
import { writeReceipt } from './ledger.js';
import { emit } from './events.js';
import { makeGateway } from './gateway.js';

// The loop every action goes through, whether it came from a dashboard click,
// an automation, or a module calling another module.
//
// request -> plan -> route -> approval? -> execute -> verify -> receipt
//
export async function request({ ctx, capability, input = {}, resource = '*', idempotencyKey }) {
  const cap = getCapability(capability);
  if (!cap) return fail(`unknown capability: ${capability}`);

  if (idempotencyKey) {
    const existing = await one(
      `select * from execution where business_id = $1 and idempotency_key = $2`,
      [ctx.businessId, idempotencyKey]
    );
    if (existing) return { execution: existing, replayed: true, result: existing. };
  }

  const disposition = await resolveDisposition({
    businessId: ctx.businessId, membership: ctx.membership, capability, resource,
  });

  if (disposition === 'never') {
    return fail(`not permitted: ${capability}`, 'forbidden');
  }

  const route = cap.route ?? 'internal';
  const plan = { capability, route, resource, disposition, steps: cap.plan?.(input) };

  const execution = await one(
    `insert into execution (business_id, requested_by, capability, input, plan, r
    values ($1,$2,$3,$4,$5,$6,$7,$8) returning *`,
    [ctx.businessId, ctx.person?.id ?? null, capability, JSON.stringify(input),
    JSON.stringify(plan), route, disposition === 'ask' ? 'awaiting_approval' : '
    idempotencyKey]
  );

  if (disposition === 'ask') {
    await one(

```

```

        `insert into approval (execution_id, business_id) values ($1,$2) returning
        [execution.id, ctx.businessId]
    );
    await emit({ businessId: ctx.businessId, topic: 'approval.requested',
        payload: { executionId: execution.id, capability } });
    return { execution, awaitingApproval: true };
}

return run({ ctx, execution, cap, input });
}

export async function decide({ ctx, executionId, decision, note = null }) {
    const execution = await one(
        `select * from execution where id = $1 and business_id = $2`, [executionId, c
    );
    if (!execution) return fail('no such execution');
    if (execution.state !== 'awaiting_approval') return fail(`execution is ${execut

    await q(
        `update approval set decided_at = now(), decided_by = $1, decision = $2, note
        where execution_id = $4`,
        [ctx.person?.id ?? null, decision, note, executionId]
    );

    if (decision !== 'approve') {
        const rejected = await setState(executionId, 'rejected');
        await emit({ businessId: ctx.businessId, topic: 'execution.rejected', payload
        return { execution: rejected, rejected: true };
    }

    await setState(executionId, 'approved');
    const cap = getCapability(execution.capability);
    const input = j(execution.input);
    return run({ ctx, execution, cap, input });
}

async function run({ ctx, execution, cap, input }) {
    await setState(execution.id, 'running');
    const gateway = makeGateway({ ctx, packageKey: cap.packageKey ?? 'kernel' });

    let result, error, verification = 'unknown', evidence = [];
    try {
        result = await cap.handler({ ctx, input, gateway, executionId: execution.id }
        await setState(execution.id, 'verifying');

        if (cap.verify) {
            const v = await cap.verify({ ctx, input, result, gateway });

```

```

        verification = v.state ?? 'unknown';
        evidence = v.evidence ?? [];
    } else {
        // No verifier means we do not get to claim the action worked.
        verification = 'unknown';
        evidence = [{ kind: 'note', text: 'capability declared no verifier' }];
    }
} catch (e) {
    error = { message: e.message };
    verification = 'failed';
    evidence = [{ kind: 'error', text: e.message }];
}

const finalState = error ? 'failed' : 'succeeded';
const updated = await one(
    `update execution set state = $1, result = $2, error = $3, updated_at = now()
    where id = $4 returning *`,
    [finalState, JSON.stringify(result ?? null), JSON.stringify(error ?? null), e
]);

const receipt = await writeReceipt({
    businessId: ctx.businessId, executionId: execution.id,
    capability: execution.capability, verification, evidence,
});

await emit({ businessId: ctx.businessId, topic: `execution.${finalState}`,
    payload: { executionId: execution.id, capability: execution.capabi

return { execution: updated, result, error, verification, receipt };
}

async function setState(id, state) {
    return one(`update execution set state = $1, updated_at = now() where id = $2 r
}

function fail(message, code = 'error') {
    return { error: { code, message } };
}

```